

Routing & State Sharing in React: App Router, React Router & Context API

MERN Stack Bootcamp Lecture

Instructor: Kashif Raza | [LinkedIn](#)

Audience: Beginner React Developers

Duration: ~90 minutes

Lecture Objectives

By the end of this session, you will be able to:

1. Understand the **difference** between Next.js App Router and React Router
2. Set up **file-based routing** (App Router) and **component-based routing** (React Router)
3. Implement **dynamic routes** to handle variable URLs like `/products/42` or `/users/kashif`
4. Use the **Context API** to share state across multiple pages without prop drilling
5. Choose the right routing & state pattern for your project

 **Key Rule:** You will **never** use App Router and React Router in the same project. Pick one based on your framework choice. This lecture covers both so you understand the ecosystem.

Routing 101: The Big Picture

Routing is how your app responds to URL changes. There are two main approaches in React:

Approach	Framework/Library	How It Works	Best For
App Router	Next.js 13+	File-based (<code>app/page.jsx</code>)	Full-stack apps, SEO, server rendering
React Router	<code>react-router-dom</code>	Component-based (<code><Routes></code>)	SPAs, client-only apps, Vite/CRA projects

Think of it like this:

- **App Router** = The file system *is* the router
- **React Router** = You write the router *inside* JavaScript components

📁 Path 1: Next.js App Router (File-Based)

Next.js uses a special `app/` directory. Every folder represents a route. Every `page.jsx` file becomes a URL endpoint.

📁 Project Structure

```
my-app/
├── app/
│   ├── layout.jsx      # Shared UI (navbar, footer)
│   ├── page.jsx        # Home route: /
│   ├── about/
│   │   └── page.jsx    # /about
│   └── contact/
│       └── page.jsx    # /contact
```

◆ `app/layout.jsx` (Wraps all pages)

```
import Link from 'next/link';

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <nav style={{ padding: '1rem', background: '#f3f4f6' }}>
          <Link href="/">Home</Link> | {' '}
          <Link href="/about">About</Link> | {' '}
          <Link href="/contact">Contact</Link>
        </nav>
        <main style={{ padding: '2rem' }}>
          {children} {/* Current page renders here */}
        </main>
      </body>
    </html>
  );
}
```

◆ `app/page.jsx` (Home Route)

```
export default function HomePage() {
  return <h1>Welcome to the App Router! 🚀</h1>;
}
```

◆ `app/about/page.jsx`

```
export default function AboutPage() {
  return <h1>About Us</h1>;
}
```

✅ **How it works:** Next.js automatically maps folders to URLs. `Link` components handle client-side navigation without full page reloads.



Path 2: React Router v6 (Component-Based)

Used in traditional React apps (Vite, Create React App). You define routes manually.



Setup

```
npm install react-router-dom
```

◆ src/App.jsx (Complete Setup)

```
import { BrowserRouter, Routes, Route, Link, useNavigate } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function Navbar() {
  const navigate = useNavigate();
  return (
    <nav style={{ padding: '1rem', background: '#f3f4f6' }}>
      <Link to="/">Home</Link> |{' '}
      <Link to="/about">About</Link> |{' '}
      <button onClick={() => navigate('/contact')}>Go to Contact (JS)</button>
    </nav>
  );
}

export default function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<h2>404 - Page Not Found</h2>} />
      </Routes>
    </BrowserRouter>
  );
}
```

✅ Key Concepts:

- `<BrowserRouter>` : Wraps your app, listens to URL changes
- `<Routes>` & `<Route>` : Match URLs to components
- `<Link>` : Navigates without reloading
- `useNavigate()` : Programmatic navigation (e.g., after form submit)

12 34 Dynamic Routing: Variable Paths

Real apps have URLs like `/products/123`, `/users/alice`, `/posts/2025/04`. You need to extract those variables.

● Next.js App Router Dynamic Routes

Create a folder with brackets: `app/products/[id]/page.jsx`

```
// app/products/[id]/page.jsx
export default async function ProductPage({ params }) {
  const { id } = await params; // Next.js 15: params is a Promise
  return <h1>Product ID: {id}</h1>;
}
```

- Visit `/products/42` → `id` is `"42"`
- Visit `/products/99` → `id` is `"99"`

● React Router Dynamic Routes

Use `:variable` syntax in path, `useParams()` to read it.

```
// src/pages/ProductDetail.jsx
import { useParams } from 'react-router-dom';

export default function ProductDetail() {
  const { id } = useParams();
  return <h1>Product ID: {id}</h1>;
}

// In your Routes:
<Route path="/products/:id" element={<ProductDetail />} />
```

✅ **Pro Tip:** Always validate dynamic params before using them (e.g., `if (!id || isNaN(id)) return <Error />`).

Context API: Share State Across Routes

The Problem: Passing `user` or `theme` down through 5 components is tedious (prop drilling).

The Solution: Context API creates a "global" value that any component can read, anywhere in the tree.

◆ Step 1: Create Context & Provider

```
// src/context/ThemeContext.jsx
import { createContext, useState, useContext } from 'react';

// 1. Create context
const ThemeContext = createContext();

// 2. Build provider component
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// 3. Custom hook for easy usage
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) throw new Error('useTheme must be used inside ThemeProvider');
  return context;
};
```

◆ Step 2: Wrap Your App

```
// src/main.jsx (or app/layout.jsx for Next.js)
import { ThemeProvider } from '../context/ThemeContext';

// React Router
<BrowserRouter>
  <ThemeProvider>
    <App />
  </ThemeProvider>
</BrowserRouter>

// Next.js
export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <ThemeProvider>{children}</ThemeProvider>
      </body>
    </html>
  );
}
```

◆ Step 3: Consume Anywhere

```
// src/components/Header.jsx
import { useTheme } from '../context/ThemeContext';

export default function Header() {
  const { theme, toggleTheme } = useTheme();

  return (
    <header style={{ background: theme === 'dark' ? '#1f2937' : 'ffffff', color: theme === 'dark'
? '#fff' : '#000' }}>
      <h1>My App</h1>
      <button onClick={toggleTheme}>Switch to {theme === 'light' ? 'Dark' : 'Light'}</button>
    </header>
  );
}
```

✅ **Why this works:** Clicking the button in `Header` updates state once. Every component using `useTheme()` re-renders automatically. No props passed down.

✔ Best Practices & Decision Guide

🌀 When to Use What

Scenario	Recommended
Building a new full-stack app with SEO & SSR	Next.js App Router
Client-only SPA, dashboard, internal tool	React Router + Vite
Sharing theme, auth, cart, locale across pages	Context API
Sharing high-frequency data (e.g., live chat, cursor)	Zustand/Jotai (Context is slow for fast updates)

⚠ Common Beginner Mistakes

```
// ❌ BAD: Putting Context inside a component that re-renders often
function App() {
  return <ThemeProvider><Router /></ThemeProvider>; // OK
}

// ✔ GOOD: Keep Provider as high as possible (usually root)
// ✔ GOOD: Split contexts (AuthContext, ThemeContext) instead of one giant object
// ✔ GOOD: Use custom hooks (useAuth, useTheme) to avoid repeating useContext
// ✔ BAD: Updating context state inside render functions (causes infinite loops)
```

🔑 Routing Checklist

- ☐ Use <Link> or next/link for navigation (never <a> for internal links)
- ☐ Always include a 404 fallback route (path="*" or app/not-found.jsx)
- ☐ Validate dynamic params before fetching data
- ☐ Keep route components focused: they should handle data fetching + UI, not business logic

Quick Practice: Build a Mini Multi-Page App

Goal: Create a 3-page app with dynamic routing and shared state.

Instructions

1. Create routes: `/` , `/about` , `/users/[id]`
2. Add a navigation bar using `<Link>` or `next/link`
3. On `/users/:id` , display "User Profile: [id]"
4. Create a `UserContext` that stores `currentUser` name
5. Add a button on the home page to "Set Current User"
6. Display the current user name on the About page using `useContext`

Starter Hints

```
// React Router example for dynamic route
<Route path="/users/:id" element={<UserPage />} />

// Next.js example structure
app/users/[id]/page.jsx

// Context usage pattern
const { userName, setUserName } = useUser();
```

Success Criteria:

- ☐ Routes work without page reload
- ☐ Dynamic ID displays correctly
- ☐ Context state updates reflect across all pages
- ☐ Clean file structure, no prop drilling

Instructor Closing

Routing and context are the backbone of real React applications. Once you understand:

- **File-based vs Component-based routing**
- **How to extract URL params**
- **How to share state without passing props manually**

...you've crossed from "tutorial React" into "production React."

Your Next Step:

Pick ONE framework for your next project. Build a 3-page app with a shared theme or auth state. Deploy it. Break it. Fix it. Repeat.

Remember: Routing handles *where* users go. Context handles *what* users see. Keep them separate, keep them clean, and your apps will scale effortlessly.

I'll be reviewing your practice repos and answering architecture questions. Tag your PRs with `#MERNBootcamp-Routing` for feedback.

— Kashif Raza | Clear paths, shared state, scalable apps 